Victor: A Local-First Research Intelligence System with Epistemic Provenance and NLI-Augmented Conflict Detection

Varish Sharma[1]*

[1]Independent Researcher
varish@example.com
https://github.com/victor-ai/victor

July 21, 2026

Abstract

Researchers accumulate knowledge across fragmented tools—papers, notes, experiments, observations—with no memory of where claims originated or how confident they should be in them. We present Victor, a local-first research intelligence system that converts raw inputs into typed, provenance-tracked Epistemic Artifacts stored in a persistent knowledge graph, answering research queries by retrieving and reasoning over those artifacts entirely on-device. Victor introduces four contributions: an Epistemic Artifact abstraction with six mandatory provenance properties, an NLI-primary architecture that eliminates structured-output agreement bias, a masked memory primitive with embedding-channel closure, and a bounded agent loop with automatic pre-answer conflict review. On the SciFact claim verification benchmark, Victor achieves 71.0% accuracy using Qwen2.5-7B and DeBERTa-v3-base NLI. Victor is open-source, CPU-only, and requires no cloud connectivity.

INTRODUCTION

The distance between a physical observation and a researcher's reliable understanding of it is enormous. A researcher today uses instruments to observe, export tools to extract data, notebooks to compute, reference managers to find papers, document viewers to read them, note apps to record thoughts, and memory to hold it all together. Every boundary between these tools destroys information. Every destroyed connection must be reconstructed from memory months later, imperfectly, or not at all.

The result is *epistemic fragmentation*: knowledge is produced but does not compound. Experiments are run but their relationship to prior claims is never recorded. Contradictions accumulate unseen. Gaps grow unnoticed. Researchers repeatedly rediscover prior insights, repeat experiments already done, and fail to build on prior work as effectively as the accumulated evidence would support.

Victor addresses this by tracking epistemic provenance from the moment a claim is created. Every piece of knowledge in Victor's graph carries a type (measurement, experiment, literature, note, or hypothesis), a confidence score derived from evidence edges, a contradiction set, a gap set, and a temporal decay rate. This structured representation enables automatic surfacing of contradictions, gaps, and patterns—and prevents the system from ever confusing a hypothesis with a measurement.

Victor is designed for deployment in environments where cloud connectivity cannot be assumed: remote field sites, research vessels, hazardous facilities. All inference, storage, and reasoning occur on-device. The system carries pre-trained models locally and operates without internet access as the default reliable state.

The key contributions of this paper are:

- **The Epistemic Artifact abstraction** — a six-property knowledge unit that enforces provenance tracking at graph construction time, enabling typed evidence aggregation and epistemic-aware retrieval scoring.
- **NLI-primary conflict detection** — an architecture that inverts the standard LLM-primary pipeline: a cross-encoder NLI model makes the direction decision while the LLM provides reasoning traces only, eliminating structured-output agreement bias.
- **Masked memory with embedding-channel closure** — a sovereignty primitive that suppresses node content from LLM answers while keeping graph structure live for routing, and deletes vector embeddings to close the membership-inference covert channel.
- **A bounded agent loop with automatic pre-answer review** — a Retrieve → Reason → Review → Respond cycle with hard tool-call limits, token ceilings, and automatic conflict detection on retrieved claims before the first LLM call.

RELATED WORK

Retrieval-Augmented Generation. Lewis et al. [2020] established the RAG paradigm: retrieve relevant documents, then generate an answer conditioned on them. Victor extends this with typed epistemic provenance—standard RAG

*Corresponding author.

has no notion of whether a retrieved chunk is a hypothesis or a measurement. Victor's retrieval scoring blends cosine similarity with importance-weighted epistemic status, ensuring higher-tier evidence ranks above lower-tier sources.

Knowledge Graph QA. KG-RAG systems [Pan et al., 2024] use knowledge graphs for entity linking and relation traversal to improve retrieval. Victor uses a knowledge graph not for entity linking but for epistemic edge tracking: CONFIRMS, CONTRADICTS, DERIVED_FROM, and GROUNDED_IN edges encode the evidential relationships between claims, experiments, and observations. This typed edge layer enables automated reasoning that flat knowledge graphs cannot support.

NLI for Fact Verification. Thorne et al. [2018] and Wadden et al. [2020] established NLI-based claim verification as a standard evaluation task. The dominant approach uses an LLM or fine-tuned classifier as the primary judge. Victor's contribution is architectural: wiring the NLI cross-encoder as the primary direction judge with the LLM as annotator-only, inverting the standard pipeline to eliminate structured-output agreement bias where instruction-tuned LLMs default to "confirm" under uncertainty.

Agentic RAG. ReAct [Yao et al., 2023], Toolformer [Schick et al., 2023], and related systems introduce tool-use loops where an LLM iteratively calls retrieval and reasoning tools. Victor bounds this loop explicitly with a hard tool-call limit (5 calls), a soft token ceiling (8,000 tokens with forced-answer fallback), and an automatic Review step that runs the conflict detector on retrieved claims with mixed evidence before the first LLM call.

Personal Knowledge Management. Tools like Obsidian, Roam Research, and Logseq capture *what* the user wrote but not *how confident* they should be in it. Victor differs by enforcing epistemic typing (every node carries an epistemic status), automated conflict detection (experiments can confirm or contradict claims), and evidence weighting (confidence scores aggregate from evidence edges).

SYSTEM DESIGN

The Epistemic Artifact

Victor's fundamental unit of knowledge is the Epistemic Artifact—a graph node with six mandatory properties:

1. **Origin** — provenance chain: sensor type and ID, source document, or extraction model.
2. **Type** — one of five epistemic tiers forming a trust hierarchy:
    - MEASUREMENT (weight 1.00): directly observed data point
    - EXPERIMENT (weight 0.85): controlled test result
    - LITERATURE (weight 0.70): claim from a published paper
    - NOTE (weight 0.50): personal annotation or summary
    - HYPOTHESIS (weight 0.25): speculative, unverified
3. **Confidence** — 0.0–1.0, computed from evidence edge aggregation.
4. **Contradictions** — set of experiment nodes with CONTRADICTS edges to this claim.
5. **Gaps** — what evidence is missing before this claim can be trusted.
6. **Age** — timestamp plus temporal decay rate for importance weighting.

The hierarchy is enforced at graph construction time: the node type determines the default epistemic status, which determines the weight used in all downstream retrieval scoring and conflict resolution. A MEASUREMENT node from a spectrometer and a HYPOTHESIS node from casual observation are structurally distinct—the system cannot confuse them.

Knowledge Graph

Victor uses a directed graph with 8 node types (PAPER, CLAIM, EXPERIMENT, NOTE, CONCEPT, CONVERSATION, COMPUTED, OBSERVATION) and 12 typed edge types. The graph is serialized as JSON via NetworkX with atomic save semantics (write to `.tmp`, then `os.replace()`) to survive crashes. Every node carries semantic embeddings computed at creation time for cosine-similarity retrieval.

Ingestion Pipeline

The ingestion layer uses a plugin architecture. Adding a new source type requires implementing `extensions` and `ingest()`—the core controller dispatches by file extension through a registry without modification. Three built-in ingestors:

- **PDFIngestor** — extracts text via PyMuPDF/pdfplumber, chunks via `TextChunker`, optionally extracts claims via LLM-based `ClaimExtractor`.
- **MarkdownIngestor** — same pipeline for `.md`/`.txt`/`.rst` files, accepts a `source_type` override.
- **ExperimentIngestor** — ingests structured JSON experiment logs, auto-links to existing claims via cosine similarity threshold.

Claims are extracted per-chunk (not on a hard-truncated document prefix) to avoid losing information from later sections. Extracted claims are deduplicated case-insensitively across chunks and stored with source provenance.

Retrieval Architecture

HybridRetriever performs two-stage retrieval with blended scoring:

$$\text{score} = 0.7 \times \text{cosine_similarity} + 0.3 \times \text{importance_score}$$

Stage 1 queries the ChromaDB vector store for top-k similar chunks. Stage 2 performs semantic graph query on node embeddings plus 2-hop neighbor expansion via typed edges. Masked nodes participate in graph routing but their content is withheld from retrieval results.

AgenticRetriever extends HybridRetriever with a bounded tool-use loop:

- **Retrieve** — hybrid vector + graph retrieval
- **Reason** — LLM calls reasoning tools (judge_conflict, summarize_claim_evidence, find_research_gaps, search_nodes_by_text, find_related_nodes, get_node_details, get_claims_for_source)
- **Review** — automatic conflict detection on retrieved claims with mixed evidence, before the first LLM call
- **Respond** — LLM generates the final answer conditioned on all gathered evidence

Hard limits: maximum 5 tool calls, 8,000 token ceiling with forced-answer fallback. Tool deduplication by SHA-256 hash prevents redundant calls.

Conflict Detection

The **ConflictDetector** determines whether an experiment confirms, contradicts, or is inconclusive with respect to a claim. It operates in two modes:

Standard mode (LLM + NLI cross-check): The LLM (Qwen2.5-7B) makes the direction decision via structured JSON output. An NLI cross-encoder (`cross-encoder/nli-deberta-v3-base`) runs afterward. If the LLM predicts confirm/inconclusive but the NLI predicts contradiction above a confidence threshold, the judgment is overridden.

NLI-primary mode: The NLI cross-encoder decides the direction. The LLM is called only for a free-form reasoning trace with no JSON schema, eliminating structured-output agreement bias.

The **NLICrossEncoder** wrapper handles label-order ambiguity across model families. Victor auto-detects the output order from `model.config.id2label`, trying three attribute paths for robustness. A pre-benchmark smoke test validates the mapping before any evaluation run.

Reasoning Layer

EvidenceWeighter walks inbound edges to a CLAIM node and computes confidence as $n_{\text{confirms}}/(n_{\text{confirms}}+n_{\text{contradicts}}+n_{\text{inconclusive}})$. **GapDetector** surfaces CLAIM nodes with no confirming experiments, sorted by importance. **PatternDetector** performs cross-gap analysis and auto-injects insights into answers when query embedding similarity to a priority gap exceeds 0.65.

Sovereignty and Security

Victor implements two orthogonal per-node control flags:

- **Freeze** prevents overwrite of node content while keeping it readable.
- **Mask** suppresses node content from LLM answers while keeping graph structure live for routing, gap detection, and conflict detection. Masked nodes emit anchor chunks preserving referential presence.

Masking also deletes the node's embedding chunks from ChromaDB, closing the membership-inference covert channel where an adversary could triangulate masked content via cosine similarity queries. Every operation is logged in a tamper-evident JSONL audit trail with caller attribution.

Computation Kernels

The **StatisticsKernel** computes descriptive statistics (mean, variance, standard deviation, confidence intervals) and Cohen's d effect size over node values. It requires an explicit `metric_name` parameter—never averaging all numeric fields, preventing conflation of distinct metrics (e.g., precision + recall + f1). Outputs are stored as COMPUTED nodes with DERIVED_FROM edges and a SHA-256 reproducibility hash.

IMPLEMENTATION

Victor is implemented in Python (v5.3.0) as a package of 16 modules totaling approximately 6,000 lines of code. The system is organized around a central `Victor` controller class that orchestrates the following components:

- **LocalEmbedder** — wraps `sentence-transformers` with a fallback chain. Primary model is nomic-embed-text-v1.5; falls back to all-MiniLM-L6-v2.
- **VictorVectorStore** — ChromaDB-backed vector store for chunk-level semantic retrieval with node-level deletion support for masked memory.
- **VictorKnowledgeGraph** — NetworkX-based directed graph serialized to JSON with atomic save semantics. Stores typed nodes and edges with all epistemic metadata.

- **InferenceClient** — provider-agnostic abstraction over Ollama (Qwen2.5-7B, primary), OpenAI API, and a mock backend for testing.
- **ToolRegistry** — 7 reasoning tools registered for the agent loop, with SHA-256 deduplication.
- **StatisticsKernel** — NumPy/SciPy-based computation with explicit metric selection.

Dependencies include `chromadb`, `sentence-transformers`, `networkx`, `ollama`, `click`, and `rich`. The system runs entirely on CPU—the largest evaluated configuration (Qwen2.5-7B Q4_K_M, approximately 5 GB) operates on consumer hardware with 32 GB RAM.

The CLI provides the complete interface via `click` and `rich`: `victor ingest`, `victor ask`, `victor gaps`, `victor observe`, `victor inspect`, `victor stats`, `victor mask`, `victor unmask`, and computation kernel commands. No web API or graph visualization is implemented—these are out of scope for the current version.

The test suite contains 196 tests across 12 test files covering the agent loop, audio pipeline, CLI imports, masked memory, masking enforcement, metric conflation, NLI cross-encoder, SciFact benchmark, sensor pipeline, statistics kernel, tool mask enforcement, and audit logging.

EVALUATION

Task and Dataset

We evaluate Victor's ConflictDetector on the SciFact benchmark [Wadden et al., 2020]: given a (claim, abstract) pair, classify the relationship as confirm, contradict, or inconclusive. The dev set contains 300 claims with evidence-linked abstracts. All experiments use n=100 samples unless otherwise noted.

Results

Table 1 summarizes the results across five configurations.

Run 1 → Run 2 regression. Forcing Qwen2.5-7B to emit valid JSON against an enum-constrained schema suppresses reasoning quality. The model routes to the lowest-perplexity token satisfying the schema—"confirm" is shorter, more common in RLHF data, and schema-valid. This produces a 10-point accuracy drop.

Run 4 failure (label-order bug). The NLI label order constant had indices 1 and 2 swapped. Every entailment prediction mapped to "inconclusive," then overridden to "contradict" by primary-mode logic. The fix auto-detects the order from `model.config.id2label` and adds a pre-benchmark smoke test.

Run 5 (current best). Cross-check mode is robust to the label-order bug because it only uses index 0 (contradiction) for the override decision. The NLI cross-encoder catches agreement bias on approximately 8 of 100 samples, adding approximately 12 points over the pure-LLM baseline.

Analysis

The 71.0% result represents the combined ceiling of Qwen2.5-7B (pure approximately 59%) plus DeBERTa-v3-base NLI cross-check. The NLI override count (5–8 of 100) indicates the cross-encoder fires infrequently—the LLM performs 92–95% of the work. Threshold tuning across values 0.3–0.6 did not improve on 71.0%, confirming that override frequency, not threshold, is the binding constraint.

Paths to 85% accuracy include: (a) a scientifically fine-tuned NLI model (SciFact-trained DeBERTa), (b) a larger LLM, (c) self-consistency voting, or (d) LoRA fine-tuning on the SciFact training split.

DISCUSSION

Agreement bias in LLM-based NLI. Instruction-tuned LLMs performing zero-shot NLI systematically default to "entailment" (confirm) when uncertain. This is training-distribution driven—RLHF data contains far more agreement than contradiction. Forcing structured JSON output amplifies this by routing to the schema-satisfying token with lowest perplexity. Victor's NLI-primary architecture addresses this by removing the direction decision from the LLM entirely, though the cross-encoder alone lacks the reasoning depth for complex scientific abstracts. The cross-check architecture—LLM decides, NLI catches agreement bias—currently achieves the best tradeoff.

Label-order ambiguity. NLI model families output label probabilities in different orders. Victor resolves this through auto-detection from the model config, an explicit `label_order` constructor parameter, and a pre-benchmark smoke test. This infrastructure caught a real failure (ModernCE's inverted order) that would have produced 35% accuracy without intervention.

Embedding covert channel. Setting a masking flag on a graph node suppresses content from LLM answers but leaves vector embeddings queryable in ChromaDB. Victor closes this channel by deleting the node's chunks from ChromaDB at mask time. This is necessary because the vector store operates independently of the graph's masking state.

LIMITATIONS

We acknowledge the following limitations:

1. **CPU-only inference limits model size.** Qwen2.5-7B is the largest model that runs comfortably on consumer hardware. Larger models or fine-tuned variants would likely improve accuracy but require GPU infrastructure.

Table 1: Conflict detection accuracy on SciFact across five configurations. Run 5 (DeBERTa cross-check) achieves the best result at 71.0%.

Run	Mode	n	Accuracy	Confirm Recall	Contradict Recall
1	Pure LLM, free-form	50	68.0%	86.0%	27.0%
2	Structured JSON	50	58.0%	63.0%	47.0%
3	Qwen2.5-7B, no NLI	100	59.0%	61.4%	53.3%
4	DeBERTa, NLI primary	100	8.0%	7.1%	10.0%
5	DeBERTa, NLI cross-check	100	**71.0%**	68.6%	76.7%

2. **Audio pipeline uses heuristic classification.** The AudioIngestor employs MFCC-based features, not deep neural network classifiers as originally planned. Classification accuracy for real-world audio events is unvalidated.

3. **No web API or graph visualization.** The system is CLI-only. Interactive graph exploration and web-based interfaces are out of scope for the current version.

4. **SciFact target not yet met.** The 85% accuracy threshold remains open. The current best of 71.0% represents the combined ceiling of the current model configuration.

5. **Evaluation on benchmark pairs only.** No real research data has been ingested and evaluated end-to-end. Validation has been on claim-abstract pairs, not full document ingestion and gap detection workflows.

6. **Single-seed evaluation at n=100.** The 71.0% result has wide confidence intervals. A full 300-sample, two-seed evaluation is required before the result can be considered stable.

CONCLUSION

Victor demonstrates that epistemic provenance tracking—typed evidence tiers, conflict detection, and sovereignty controls—can be implemented as a coherent local-first system without cloud dependency. The Epistemic Artifact abstraction provides a structured foundation for research knowledge management beyond flat retrieval or linked notes.

The NLI-primary architecture identifies and addresses structured-output agreement bias in LLM-based fact verification. While the current cross-encoder lacks reasoning depth for primary-mode deployment, the cross-check configuration achieves 71.0% on SciFact with clear paths to improvement.

The masked memory primitive with embedding-channel closure establishes that content suppression and graph routing can coexist—and that the vector embedding covert channel must be explicitly closed, not just flagged.

Future work targets the 85% SciFact threshold via fine-tuned NLI models, followed by MCP server integration for distribution through Claude Desktop and Cursor, and ultimately the physical device platform with multi-sensor fusion.

REFERENCES

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7): 2958–2978, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2018.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. SciFact: A benchmark for scientific claim verification. *arXiv preprint arXiv:2005.10645*, 2020.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Ilya Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In

International Conference on Learning Representations (ICLR), 2023.